

CANDY DIALOGUE CREATOR

In-Game Integration

1.0.0

At this stage, what follows is still somewhat of an experimental feature and may not work properly in all cases. It can also require varying degrees of code modification on your part. It's theoretically feasible, but largely untested.

Currently, this guide only explains the basic principles of integrating Candy Dialogue Creator into your Godot project: the specific details are left entirely up to you, and will require that you have some experience with how Candy Dialogue Engine works. In time, we may provide more detailed guides and tutorials.

The Candy Dialogue Creator can be integrated into your game project, such that players can access it from there at runtime.

This is not required or expected, but you may want to do it for the following reasons:

- 1) Give yourself, as a developer, the option to write and test dialogues live at runtime.
- 2) Enable players to write dialogues at runtime, and store them in native dictionaries rather than external files (see Candy Dialogue Engine's [Modding](#) guide).

In the second case, this allows you to customize the Candy Dialogue Creator, such as making specific commands unavailable for 'mod dialogues'.

Basic Integration

Integrating the Candy Dialogue Creator as-is is very simple:

1. Copy the "**Candy_DC**" folder to your project folder.
2. Set the "**User Variables.gd**" script as an autoload in your project, with the prefix 'candy_dc'.
3. Add the "**Main UI.tscn**" scene to a scene in your game, or make your game load it at runtime.
4. In your game, setup a way to access the "Main UI" scene (e.g. a button in a menu).

Important note: the UI was designed for a screen resolution of 1920x1080. If your project uses a smaller resolution, you may need to resize some UI elements. Offering more flexibility with resolutions is part of our future plans.

Once you've completed the four steps above, you'll probably want to modify the code to prevent the Project Menu from appearing on-screen immediately. For example, set it to invisible by default, and set it to visible when the Candy Dialogue Creator is accessed within your game.

You might also want to modify the code called by the "Exit" button (top-left corner of the Main UI) to return to your game's main menu rather than quitting the game entirely.

And you might also want to block players from accessing some parts of the Settings Menu, and to setup specific paths for project folders, since in all likelihood you'll want this integration of the Candy Dialogue Creator to create dialogues for your game only.

Export to Native Dictionaries

If you want dialogues to be saved to native dictionaries, you'll need to modify the Export Menu to save to such dictionaries instead of saving to external .txt files (you should also modify the Test button, or remove it entirely).

Likewise for the Import Menu, which should import from those same dictionaries.

Restricting Commands

If you want to ensure some commands aren't available you should probably disable or modify the Save and Load features. Since Candy Dialogue Creator saves to/loads from .txt files by default, users could create such save files, modify them externally to add forbidden commands, then load them back in.

Besides that, blocking access to commands is as simple as deleting their buttons in the Main UI.

Keep in mind that the \$Set command can modify literally any variable in your game. As a result, it could be used to add any commands into dialogues, and it could even modify variables in your game's integration of the Candy Dialogue Editor.

The best way to ensure specific commands aren't available to player-created dialogues would be to use a unique instance of the dialogue engine, with a modified Candy_Engine.gd script: that modified script would have all prohibited commands removed. However, you would need to remove or modify the \$Call command as well, as it could otherwise be used to run dialogues on a normal instance of the dialogue engine (i.e. an instance with all commands available).

Further Integration

And of course, further modifications may be required if you have other requirements. Integrating the Candy Dialogue Creator as-is to your game isn't difficult at all, but customizing that integration can add some challenge. So while this is very much a feature of Candy Dialogue Creator, you should ask yourself if integrating it to your game is the right call:

- What are you trying to achieve with an in-game integration vs. letting players use the regular standalone version?
- What do you actually need to customize to fit your purposes?